

Insurance AI Application Lab Guide

LangChain · Docling · Streamlit · OpenAI

AWS EC2 Ubuntu | Code-Server IDE | Python 3.10+
Fresher-Level Bootcamp | 5 Hands-On Labs

Lab	Topic	Technology Stack
Lab 1	LLM Chain for Insurance Query Handling	LangChain
Lab 2	Chatbot with Conversation Memory	LangChain Memory
Lab 3	Insurance Policy PDF Extraction	Docling
Lab 4	Streamlit Chatbot UI	Streamlit
Lab 5	Full-Stack Chatbot: UI + Memory + Documents	LangChain · Streamlit · Docling

■ Prerequisites	■ Estimated Duration
• AWS EC2 Ubuntu instance (t3.medium or higher) • Code-Server IDE accessible via browser • Python 3.10+ installed • OpenAI API Key available • pip and virtualenv installed	• Lab 1: ~30 minutes • Lab 2: ~30 minutes • Lab 3: ~40 minutes • Lab 4: ~30 minutes • Lab 5: ~60 minutes Total: ~3.5 hours

Table of Contents

Environment Setup	Global setup before any lab	p. 3
Lab 1 — LLM Chain for Insurance Queries	LangChain basics: prompts, chains, output parsers	p. 5
Lab 2 — Chatbot with Conversation Memory	ConversationBufferMemory, chat history management	p. 9
Lab 3 — PDF Extraction with Docling	Load, parse, and structure insurance policy PDFs	p. 13
Lab 4 — Streamlit Chatbot UI	Build an interactive chat interface with Streamlit	p. 17
Lab 5 — Full-Stack Insurance Chatbot	Combine UI + Memory + Document Ingestion	p. 21
Troubleshooting & Quick Reference	Common errors and fixes	p. 27

Environment Setup

One-time configuration before all labs

Overview

All labs run inside a Python virtual environment on your AWS EC2 Ubuntu instance. The Code-Server IDE gives you a browser-based VS Code experience. Follow these steps once before starting Lab 1.

Step 1 — Open a Terminal in Code-Server

Open your browser and navigate to your EC2 public IP on port 8080 (e.g., <http://<EC2-IP>:8080>). In Code-Server, open a new terminal via **Terminal** → **New Terminal**.

Note

If prompted for a password, use the one set during Code-Server installation.

Step 2 — Create Project Directory & Virtual Environment

```
# Create working directory
mkdir ~/insurance ai lab && cd ~/insurance ai lab

# Create virtual environment
python3 -m venv venv

# Activate it
source venv/bin/activate

# Confirm activation (should show venv path)
which python
```

Step 3 — Install All Required Packages

```
# Upgrade pip first
pip install --upgrade pip

# Core packages for all labs
pip install langchain langchain-openai langchain-community
pip install openai
pip install docling
pip install streamlit
pip install python-dotenv
pip install faiss-cpu
pip install tiktoken
```

Step 4 — Configure OpenAI API Key

Create a `.env` file to store your OpenAI API key securely. Never hardcode keys in your Python files.

```
# Create .env file
cat > .env << EOF

OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

EOF
```

```
# Verifv the file
cat .env
```

Warning

Replace sk-xxxx... with your actual OpenAI API key provided by the instructor.

Step 5 — Project Folder Structure

Create sub-directories for each lab:

```
mkdir -p lab1 lab2 lab3 lab4 lab5 lab3/sample pdfs
ls -la
```

insurance_ai_lab/

```
■■■ .env
■■■ venv/
■■■ lab1/ ← LLM Chain
■■■ lab2/ ← Memory Chatbot
■■■ lab3/ ← Docling PDF Extraction
    ■■■ sample_pdfs/
■■■ lab4/ ← Streamlit UI
■■■ lab5/ ← Full-Stack Chatbot
```

Ready

Your environment is now configured. Keep the terminal open and proceed to Lab 1.

Lab 1 — LLM Chain for Insurance Query Handling

Build a simple chain to answer insurance questions using LangChain

■ Lab Objectives

- Understand the LangChain chain architecture
- Create a prompt template for insurance Q&A
- Connect a prompt template to an OpenAI LLM
- Parse and display structured LLM responses
- Test with real insurance queries

Concept Overview

LangChain chains connect a **PromptTemplate** → **LLM** → **OutputParser** in a single pipeline. The prompt template defines how we phrase insurance questions to the model, the LLM generates a response, and the output parser formats it for our app.



Step 1 — Create the Python File

```
cd ~/insurance_ai_lab/lab1
touch insurance_chain.py
# Open in Code-Server editor
code insurance_chain.py
```

Step 2 — Import Libraries and Load Environment

```
# insurance_chain.py

import os
from dotenv import load_dotenv
from langchain.openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Load API key from .env
load_dotenv(dotenv_path="/root/insurance_ai_lab/.env")
```

Step 3 — Define the Prompt Template

The prompt template sets the AI's role as an insurance expert and injects the user's query. The **{query}** placeholder will be filled at runtime.

```
# Define system and human message prompts
prompt = ChatPromptTemplate.from_messages([
    ("system",
```

```
"You are a knowledgeable insurance advisor for an Indian insurance company. "
"Answer questions about health, life, motor, and property insurance. "
"Be concise, accurate, and professional. If unsure, say so."),
("human", "{query}"),
])
```

Step 4 — Initialize the LLM and Build the Chain

```
# Initialize the LLM (gpt-4o-mini is cost-effective for labs)
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.3,
    openai_api_key=os.getenv("OPENAI_API_KEY")
)

# Build the chain using LCEL (LangChain Expression Language)
output_parser = StrOutputParser()
chain = prompt | llm | output_parser

# The | operator pipes: prompt → llm → parser
```

Step 5 — Run Multiple Insurance Queries

```
# Test with insurance-specific queries
queries = [
    "What is the difference between term life and whole life insurance?",
    "What is not covered under a standard health insurance policy in India?",
    "How is the premium for motor insurance calculated?",
    "What is the claim settlement ratio and why does it matter?",
]

for i, query in enumerate(queries, 1):
    print(f"\n" + "="*60)
    print(f"Query {i}: {query}")
    print("="*60)
    response = chain.invoke({"query": query})
    print(response)
```

Step 6 — Run the Script

```
python insurance_chain.py
```

■ Expected Output

You should see structured insurance answers for each query printed to the terminal. Each response will reflect the insurance expert persona defined in the prompt.

■ Challenge Task

Modify the prompt to also respond in Hindi alongside English. Try using **temperature=0.7** and observe how responses change. Add a new query: *"What is the IRDAI and what does it regulate?"*

Lab 2 — Chatbot with Conversation Memory

Maintain chat history so the bot remembers previous messages

■ Lab Objectives

- Understand how LLMs are stateless by default
- Use **ConversationBufferMemory** to store chat history
- Build a conversational chain with **ConversationChain**
- Test multi-turn conversations about insurance policies
- Inspect and clear memory programmatically

Concept: Why Memory Matters

An LLM processes each API call independently — it has no memory of previous messages. LangChain's **ConversationBufferMemory** maintains a rolling window of the conversation and automatically injects it into every new request, making multi-turn conversations possible.



Step 1 — Create the File

```
cd ~/insurance_ai_lab/lab2
touch memory_chatbot.py
code memory_chatbot.py
```

Step 2 — Imports and Setup

```
import os
from dotenv import load_dotenv
from langchain.openai import ChatOpenAI
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
from langchain.core.prompts import PromptTemplate
load_dotenv(dotenv_path="/root/insurance_ai_lab/.env")
```

Step 3 — Custom Prompt with Memory Placeholder

The **{history}** placeholder is automatically filled by LangChain with previous conversation turns. The **{input}** placeholder receives the new user message.

```
INSURANCE_TEMPLATE = """
You are an expert insurance advisor at InsureSafe India.
You help customers understand policies, claims, premiums, and coverage.
Always be helpful, clear, and professional.
Previous Conversation:
```

```
{history}

Customer: {input}
InsureSafe Advisor:""

prompt = PromptTemplate(
    input_variables=["history", "input"],
    template=INSURANCE_TEMPLATE
)
```

Step 4 — Initialize LLM, Memory, and Chain

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.4,
    openai_api_key=os.getenv("OPENAI_API_KEY")
)

# Memory stores the rolling conversation
memory = ConversationBufferMemory(
    memory_key="history",
    return_messages=False
)

chain = ConversationChain(
    llm=llm,
    memory=memory,
    prompt=prompt,
    verbose=True # Shows full prompt including history
)
```

Step 5 — Run a Multi-Turn Conversation

```
# Simulate a realistic multi-turn insurance conversation
conversation = [
    "Hi, I want to buy health insurance for my family of 4.",
    "What is the minimum sum insured I should go for?",
    "Does it cover pre-existing conditions like diabetes?",
    "How long is the waiting period for that?",
    "Can you summarize what we discussed so far?",
]

for turn, user_msg in enumerate(conversation, 1):
    print(f"\n--- Turn {turn} ---")
    print(f"Customer: {user_msg}")
    response = chain.predict(input=user_msg)
    print(f"Advisor: {response}")
```

Step 6 — Inspect and Clear Memory


```
# View what is stored in memory
print("\n=== Memory Buffer ===")
print(memory.buffer)

# Clear memory (useful for starting new conversations)
memory.clear()
print("Memory cleared! Buffer:", memory.buffer)
```

Step 7 — Run the Script

```
python memory_chatbot.py
```

■ Expected Output

You should see 5 conversation turns. Notice that Turn 5 ('Can you summarize...') correctly references details from earlier turns — proof that memory is working.

■ Challenge Task

Switch to **ConversationBufferWindowMemory(k=3)** which only keeps the last 3 turns. Observe how the summary response (Turn 5) changes. This is useful for controlling token costs in production.

Lab 3 — Insurance Policy PDF Extraction with Docling

Load, parse, and extract structured data from insurance PDF documents

■ Lab Objectives

- Install and configure Docling for PDF processing
- Create a sample insurance policy PDF for testing
- Use Docling to load and parse the PDF document
- Extract structured sections: coverage, exclusions, premiums
- Export extracted data to JSON and Markdown formats

What is Docling?

Docling is an open-source document understanding library from IBM that converts PDFs, Word documents, and other files into structured formats. It uses advanced ML models to understand document layout, tables, headings, and paragraphs — far superior to basic text extraction tools for complex insurance documents.

Step 1 — Navigate to Lab Directory

```
cd ~/insurance_ai_lab/lab3
```

Step 2 — Create a Sample Insurance Policy PDF

Since we need a PDF to test with, we'll generate one using Python. Create a file called `create_sample_pdf.py`:

```
# create sample pdf.py
from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.lib import colors

doc = SimpleDocTemplate("sample_pdfs/health_policy.pdf", pagesize=letter)
styles = getSampleStyleSheet()
story = []

story.append(Paragraph("InsureSafe Health Policy - Gold Plan", styles["Title"]))
story.append(Paragraph("Policy Number: ISH-2024-GOLD-001", styles["Normal"]))
story.append(Spacer(1, 12))

story.append(Paragraph("Coverage Details", styles["Section-Header"]))
coverage = [
    ["Benefit", "Coverage Amount", "Notes"],
    ["Sum Insured", "Rs. 10,00,000", "Per Policy Year"],
    ["Hospitalisation", "Up to SI", "Min 24 hours admission"],
    ["Day Care", "Up to SI", "540+ listed procedures"],
    ["Pre-hospitalisation", "60 days", "Related expenses covered"],
    ["Post-hospitalisation", "90 days", "Related expenses covered"],
```

```

1
t = Table(coverage)
t.setStyle(TableStyle([
("BACKGROUND", (0,0), (-1,0), colors.grev),
("GRID", (0,0), (-1,-1), 1, colors.black),
]))
story.append(t)

story.append(Paragraph("Exclusions", styles["Heading1"]))
exclusions = [
"1. Pre-existing conditions: 3-year waiting period applies.",
"2. Cosmetic or plastic surgery not medically necessary.",
"3. Dental treatment unless arising from accident.",
"4. Maternity expenses in first 2 policy years.",
]
for ex in exclusions:
story.append(Paragraph(ex, styles["Normal"]))
doc.build(story)
print("Sample PDF created!")

pip install reportlab --quiet
python create_sample_pdf.py

```

Step 3 — Write the Docling Extraction Script

Create `pdf_extractor.py`:

```

# pdf_extractor.py
import ison
import os
from pathlib import Path
from docling.document_converter import DocumentConverter
from docling.datamodel.base_models import InputFormat
from docling.datamodel.pipeline_options import PdfPipelineOptions
from docling.document_converter import PdfFormatOption

```

Step 4 — Configure Docling Pipeline

```

# Configure pipeline options
pipeline_options = PdfPipelineOptions()
pipeline_options.do_ocr = False # True for scanned PDFs
pipeline_options.do_table_structure = True # Extract tables

# Initialize the converter
converter = DocumentConverter(
    format_options={
        InputFormat.PDF: PdfFormatOption(pipeline_options=pipeline_options)
    }
)

```

```
)

# Set PDF path
pdf_path = Path("sample_pdfs/health_policy.pdf")
```

Step 5 — Convert and Extract Content

```
print(f"Processing: {pdf_path.name}")
result = converter.convert(str(pdf_path))

# Get the Docling document object
doc = result.document

# Export to Markdown (preserves headings, tables, lists)
markdown_output = doc.export_to_markdown()
print("\n=== Markdown Output ===")
print(markdown_output)

# Save markdown to file
with open("extracted_policy.md", "w") as f:
    f.write(markdown_output)
print("\nSaved to extracted_policy.md")
```

Step 6 — Extract Structured Data with OpenAI

Now use the extracted text as context to query OpenAI for structured extraction:

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv(dotenv_path="/root/insurance_ai_lab/.env")
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

extraction_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Extract structured data from this insurance policy. "
     "Return JSON with keys: policy name, sum insured, exclusions[], "
     "benefits[], waiting periods[]. No extra text, just JSON."),
    ("human", "Policy Document:\n{document}"),
])

chain = extraction_prompt | llm | StrOutputParser()
result_json = chain.invoke({"document": markdown_output})
parsed = json.loads(result_json)
print("\n=== Extracted Structure ===")
print(json.dumps(parsed, indent=2))
```

```
python pdf_extractor.py
```

■ Expected Output

The script will print the full policy in Markdown format, then output a clean JSON structure with policy_name, sum_insured, exclusions list, benefits list, and waiting periods.

■ Challenge Task

Create a second PDF for a Motor Insurance policy, run extraction on both, and compare the structured outputs side-by-side. Save results as **motor_policy.json**.


```
"Select Model",
["apt-4o-mini", "apt-4o"]
)
temperature = st.slider("Temperature", 0.0, 1.0, 0.3)
st.divider()
if st.button("■ Clear Chat"):
st.session_state.messages = []
st.rerun()
```

Step 4 — Chat Interface

```
# █ Main Chat Area
st.title("█ InsureSafe AI Assistant")
st.caption("Ask me anything about insurance policies, claims, and coverage")
# Initialize session state for messages
if "messages" not in st.session_state:
    st.session_state.messages = [
        {
            "role": "assistant",
            "content": "Hello! I'm your InsureSafe advisor. How can I help you today?"
        }
    ]
# Display all previous messages
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])
```

Step 5 — Handle User Input and LLM Response

```
# Handle new user input
if user input := st.chat_input("Type your insurance question here..."):
    # Append user message
    st.session_state.messages.append({
        "role": "user", "content": user input
    })
    with st.chat_message("user"):
        st.markdown(user input)

    # Build message history for LLM
    llm = ChatOpenAI(model=model, temperature=temperature,
        openai_api_key=os.getenv("OPENAI_API_KEY"))

    lc.messages = [HumanMessage(content="You are an expert insurance advisor.")]
    for m in st.session_state.messages:
        if m["role"] == "user":
            lc.messages.append(HumanMessage(content=m["content"]))
        else:
            lc.messages.append(AIMessage(content=m["content"]))

    # Stream response to UI
```

```
with st.chat_message("assistant"):
    response = st.write_stream(llm.stream(messages))
    st.session_state.messages.append({
        "role": "assistant", "content": response
    })
```

Step 6 — Launch Streamlit



Code-Server Port

Streamlit runs on port 8501 by default. Access it at <http://:8501> in your browser. Make sure port 8501 is open in your EC2 Security Group.

```
# Run from lab4 directory
streamlit run streamlit_app.py --server.port 8501 --server.address 0.0.0.0
```

UI Running

Open <http://:8501> in a new browser tab. You should see the InsureSafe AI chatbot with a professional chat interface, sidebar controls, and streaming responses.

Challenge Task

Add a message counter in the sidebar showing "Total messages: X". Also add a download button that exports the chat history as a .txt file using **`st.download_button`**.

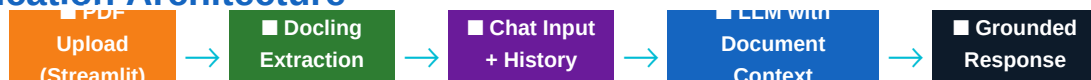
Lab 5 — Full-Stack Insurance Chatbot

Combine Streamlit UI + LangChain
Memory + Docling Document Ingestion

■ Lab Objectives

- Build a complete insurance chatbot application
- Enable PDF upload directly in the Streamlit UI
- Process uploaded PDFs with Docling in real-time
- Ground LLM responses in the extracted policy documents
- Maintain multi-turn conversation memory across the session
- Display document context alongside chat responses

Application Architecture



Step 1 — Setup

```
cd ~/insurance ai lab/lab5
mkdir -p tmp uploads
touch full_chatbot.py
code full_chatbot.py
```

Step 2 — Imports and Configuration

```
# full_chatbot.py
import os, tempfile
from pathlib import Path
import streamlit as st
from dotenv import load_dotenv
from langchain.openai import ChatOpenAI
from langchain.core.messages import HumanMessage, AIMessage, SystemMessage
from docling.document_converter import DocumentConverter

load_dotenv(dotenv_path="/root/insurance ai lab/.env")

st.set_page_config(
    page_title="InsureSafe Pro",
    page_icon="🏠",
    layout="wide"
)
```

Step 3 — Session State Initialization

```
# Initialize all session state variables
def init_session():
    defaults = {
```

```

"messages": [],
"policy context": "",
"policy name": None,
"doc loaded": False,
}
for key, val in defaults.items():
    if key not in st.session_state:
        st.session_state[key] = val
init_session()

```

Step 4 — Docling PDF Processing Function

```

@st.cache_data # Cache so same file is not re-processed
def extract_policy_text(pdf_bytes: bytes, filename: str) -> str:
    """Extract text from PDF using Docling."""
    with tempfile.NamedTemporaryFile(
        suffix=".pdf", delete=False
    ) as tmp:
        tmp.write(pdf_bytes)
        tmp_path = tmp.name

    try:
        converter = DocumentConverter()
        result = converter.convert(tmp_path)
        text = result.document.export_to_markdown()
    finally:
        os.unlink(tmp_path) # Clean up temp file

    return text

```

Step 5 — Sidebar: Upload and Settings

```

with st.sidebar:
    st.header("■ Upload Policy Document")
    uploaded = st.file_uploader(
        "Upload an insurance policy PDF",
        type=["pdf"],
        help="The chatbot will answer questions based on this document"
    )

    if uploaded and uploaded.name != st.session_state.policy_name:
        with st.spinner("■ Reading policy document..."):
            pdf_text = extract_policy_text(uploaded.read(), uploaded.name)
            st.session_state.policy_context = pdf_text
            st.session_state.policy_name = uploaded.name
            st.session_state.doc_loaded = True
            st.session_state.messages = []
            st.success(f"■ Loaded: {uploaded.name}")

    if st.session_state.doc_loaded:

```

```

st.info(f"■ Active: {st.session_state.policy name}")
st.divider()
model = st.selectbox("Model", ["gpt-4o-mini", "gpt-4o"])
temperature = st.slider("Temperature", 0.0, 1.0, 0.3)
if st.button("■ Clear Chat"):
    st.session_state.messages = []
    st.rerun()

```

Step 6 — Main Chat Area with Document-Grounded Responses

```

col1, col2 = st.columns([3, 1])
with col1:
    st.title("■ InsureSafe Pro Advisor")
    # System prompt grounded in uploaded document
    if st.session_state.doc_loaded:
        system_msg = SystemMessage(content=
            f"You are an expert insurance advisor. Use ONLY the policy document below "
            f"to answer questions. If something is not in the document, say so.\n\n"
            f"POLICY DOCUMENT:\n{st.session_state.policy context[:6000]}")
    )
    st.caption(f"■ Grounded on: {st.session_state.policy name}")
    else:
        system_msg = SystemMessage(content=
            "You are an expert insurance advisor. Answer general insurance questions."
        )
    st.caption("■ No document uploaded – answering from general knowledge")

```

Step 7 — Display Messages and Handle Input

```

# Display chat history
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

if user_input := st.chat_input("Ask about this policy..."):
    st.session_state.messages.append({
        "role": "user", "content": user_input
    })

    with st.chat_message("user"):
        st.markdown(user_input)

# Build LangChain message list with history
lc_msgs = [system_msg]
for m in st.session_state.messages:
    cls = HumanMessage if m["role"] == "user" else AIMessage
    lc_msgs.append(cls(content=m["content"]))

llm = ChatOpenAI(model=model, temperature=temperature,

```

```
openai api key=os.getenv("OPENAI_API_KEY"))
with st.chat_message("assistant"):
    response = st.write_stream(llm.stream(messages))
    st.session_state.messages.append({
        "role": "assistant", "content": response
    })
```

Step 8 — Right Column: Document Preview

```
with col2:
    st.subheader("■ Policy Preview")
    if st.session_state.doc_loaded:
        # Show first 800 chars as preview
        preview = st.session_state.policy_context[:800]
        st.text_area("Extracted Text", preview, height=400)
        st.metric("Doc Length",
            f"{len(st.session_state.policy_context):,} chars")
    else:
        st.info("Upload a PDF to see preview")
```

Step 9 — Launch the Full Application

```
streamlit run full_chatbot.py --server.port 8502 --server.address 0.0.0.0
```

■ Full App Running

Navigate to <http://:8502>. Upload the `health_policy.pdf` from Lab 3. Ask: 'What is excluded from this policy?' — the bot will answer from the document!

Testing Scenarios

Scenario	Test Query	Expected Behavior
Document Q&A;	"What is the sum insured?"	Answers from uploaded PDF
Memory Test	"What was my first question?"	References earlier turn in chat
Out-of-scope	"What is the stock market?"	Bot says not in the document
No doc mode	"Explain term insurance"	Uses general knowledge

■ Final Challenge Task

Add a **Summary tab** using `st.tabs(["Chat", "Policy Summary"])`. In the Summary tab, display an LLM-generated summary of the uploaded policy with key fields like coverage, exclusions, premiums, and waiting periods formatted as a table. Trigger summary generation when the PDF is first uploaded.

Troubleshooting & Quick Reference

Common errors, fixes, and command reference for all labs

■ Error: AuthenticationError: Incorrect API key

Cause: Your OPENAI_API_KEY in .env is invalid or not loaded.

Fix: Verify with: `python -c "import os; from dotenv import load_dotenv; load_dotenv('/root/insurance_ai_lab/.env'); print(os.getenv('OPENAI_API_KEY')[:8])"`

■ Error: ModuleNotFoundError: No module named 'langchain'

Cause: Packages are not installed or venv is not activated.

Fix: Run: `source ~/insurance_ai_lab/venv/bin/activate && pip install langchain langchain-openai`

■ Error: Streamlit: port 8501 already in use

Cause: Another Streamlit process is running.

Fix: Run: `pkill -f streamlit` or use `--server.port 8503`

■ Error: Docling: ImportError or model download fails

Cause: Docling downloads ML models on first run. May need internet access.

Fix: Check EC2 security group allows outbound HTTPS. Run docling in a test script first.

■ Error: RateLimitError from OpenAI

Cause: You have exceeded your API quota or per-minute limit.

Fix: Add `time.sleep(1)` between requests. Upgrade your OpenAI plan if needed.

■ Error: Streamlit: StreamlitAPIException - chat_input in sidebar

Cause: `st.chat_input` cannot be placed inside `st.sidebar`.

Fix: Move `st.chat_input` to the main content area, not inside `'with st.sidebar:'` block.

■ Error: ConversationChain: KeyError 'history'

Cause: Memory key mismatch between `ConversationBufferMemory` and `PromptTemplate`.

Fix: Ensure `memory_key='history'` in `ConversationBufferMemory` matches `{history}` in the template.

Quick Command Reference

Action	Command
Activate venv	<code>source ~/insurance_ai_lab/venv/bin/activate</code>

Deactivate venv	<code>deactivate</code>
Check installed packages	<code>pip list grep -E "langchain docling streamlit"</code>
Run Lab 1	<code>cd ~/insurance_ai_lab/lab1 && python insurance_chain.py</code>
Run Lab 2	<code>cd ~/insurance_ai_lab/lab2 && python memory_chatbot.py</code>
Run Lab 3	<code>cd ~/insurance_ai_lab/lab3 && python pdf_extractor.py</code>
Run Lab 4 UI	<code>cd ~/insurance_ai_lab/lab4 && streamlit run streamlit_app.py --server.address</code>
Run Lab 5 UI	<code>cd ~/insurance_ai_lab/lab5 && streamlit run full_chatbot.py --server.port 8</code>
Kill Streamlit	<code>pkill -f streamlit</code>
Check API Key loaded	<code>python -c "from dotenv import load_dotenv; import os; load_dotenv('~/.insura</code>

Key Concepts Summary

Concept	Description	Used In
LCEL Chain	<code>prompt llm parser</code> pipeline syntax	Labs 1, 3
PromptTemplate	Reusable prompt with {variable} placeholders	Labs 1, 2
ConversationBufferMemory	Stores full chat history in memory	Lab 2
ConversationChain	LLM chain with integrated memory management	Lab 2
DocumentConverter (Docling)	Converts PDFs to structured Markdown/JSON	Labs 3, 5
<code>st.session_state</code>	Streamlit persistent state across reruns	Labs 4, 5
<code>st.chat_input</code> / <code>st.chat_message</code>	Streamlit native chat UI components	Labs 4, 5
SystemMessage	LangChain message setting LLM behavior/context	Labs 4, 5
<code>st.write_stream</code>	Streams LLM tokens to Streamlit UI in real-time	Labs 4, 5

■ **Bootcamp Complete!** You have built a full-stack AI-powered insurance advisor using LangChain, Docling, and Streamlit. The skills learned here — chaining LLMs, managing memory, parsing documents, and building UIs — form the core of real-world production AI applications.